The document with this ID should be picked up from the `classic_movies` index, because the `index` field mentions that ID in the query. And of course, the field from which to fetch the values is `director`, as declared as the `path` field. Running this query fetches two movies that were directed by Spielberg.

The `terms` lookup query helps build a query based on values obtained from another document rather than a set of values passed in the query. It offers greater flexibility when constructing query terms: we can easily swap the index with any other index from which to obtain a document. For example, say we have an index called `movie_search_terms_index` containing several documents of search terms (document 1 contains `director` terms, document 2 contains `actors` terms, and so on). We can reference this document with `director` terms from `movie_search_terms_index` in our main query and fetch the results. This way, the main query can be constant while the lookup documents are changed as required. Now that we understand `terms` queries, let's move on to a query type where we fetch documents given a set of IDs.

9.4 The ids query

Sometimes we want to obtain the documents that have those IDs from Elasticsearch. As the name suggests, the `ids` query fetches matching documents given a set of document IDs. It's a much simpler way to fetch multiple documents simultaneously. The following listing shows how to retrieve documents using a list of document IDs.

Listing 9.9 Fetching multiple documents using an `ids` query

```
GET movies/_search
{
  "query": {
    "ids": {
      "values": [10,4,6,8]
    }
  }
}
```

Name of the query

Provides the document IDs as an array

This query returns four documents with the corresponding four IDs. Each document that is indexed has a mandatory `_id` field.

> **NOTE** Metadata fields are not allowed to be part of the schema definition. The `_id` field, along with other fields like `_source`, `_size`, `_routing`, and so on, are metadata fields and so cannot be part of the index mapping exercise.

We can also use a `terms` query to fetch documents with a set of document IDs, instead of using an `ids` query, as we saw in listing 9.4. The next listing shows another example.

Listing 9.10 A `terms` query using a set of IDs to fetch documents

```
GET movies/_search
{
  "query": {
    "terms": {
```